Ray Tracing

Radosław Szczepaniak

June 9, 2026

1 Funkcja `main`

Funkcja `main` wywołuje `build_scene`, która tworzy wszystkie obiekty i światła i zwraca gotowy obiekt `Scene`. Następnie przygotowuje bufor obrazu - jeden element typu `Color` na każdy piksel - i wchodzi w dwie pętle przebiegające po wierszach i kolumnach obrazu.

Dla każdego piksela kod oblicza jego położenie na płaszczyźnie ekranu, tworzy obiekt `Ray` prowadzący od kamery przez ten punkt i przekazuje go do funkcji `ray_color`. Robi to w pętli `samples_per_pixel` razy, za każdym razem z drobnym losowym przesunięciem celu wewnątrz piksela; zwrócone kolory są sumowane, a po wyjściu z pętli dzielone przez liczbę próbek. Uśredniona wartość trafia do `framebuffer`. To właśnie wielokrotne próbkowanie realizuje antyaliasing – w kodzie sprowadza się ono po prostu do powtórzenia wywołania `ray_color` i uśrednienia wyników.

Zewnętrzna pętla po wierszach poprzedzona jest dyrektywą `#pragma omp parallel for`, dzięki czemu poszczególne wiersze obrazu liczone są równolegle na wielu wątkach.

2 Reprezentacja danych

Informacje krążące w programie opisują trzy proste typy. Klasa `Vec3` przechowuje trzy liczby i służy jednocześnie jako punkt, wektor i kolor (alias `Color`) – dzięki temu te same operatory (`+`, `*`, `dot`, `cross`) używane są zarówno do geometrii, jak i do mieszania barw. Klasa `Ray` to para: punkt początkowy i kierunek, z metodą `at(t)` zamieniającą parametr na konkretny punkt. Strukturę `HitRecord` kod wykorzystuje zapisując w niej punkt przecięcia, wektor normalny, parametr `t`, informację o stronie powierzchni oraz wskaźnik na materiał. Ta struktura przekazywana jest przez referencję.

3 Funkcja `ray_color`

Najwięcej dzieje się w rekurencyjnej funkcji `ray_color`, która dla zadanego promienia zwraca jego barwę.

- na wejściu sprawdza licznik `depth`; gdy spadnie do zera, funkcja natychmiast zwraca czerń, co przerywa rekurencję;
- wywołuje `scene.hit(...)`, aby ustalić, czy i gdzie promień trafia w scenę; jeśli nie - zwraca wynik `background(r)`, czyli barwę tła;
- po trafieniu pyta materiał trafionego obiektu przez `rec.mat->scatter(...)`; ta funkcja wypełnia dwie zmienne wyjściowe: `attenuation` (kolor pochłaniania) oraz `scattered` (nowy promień);
- jeśli materiał wytworzył promień wtórny, funkcja *wywołuje samą siebie* dla `scattered` z licznikiem zmniejszonym o jeden, a otrzymaną barwę mnoży przez `attenuation` i dodaje do niej oświetlenie bezpośrednie z `direct_light`.

W wywołaniu rekurencyjnym zawiera się cała mechanika wielokrotnych odbić.

4 Jak kod znajduje widoczny obiekt

Funkcja `Scene::hit` nie zna konkretnych kształtów – przechodzi po wektorze obiektów i dla każdego woła jego własną metodę `hit`. Aby spośród wszystkich trafień wybrać to widoczne, utrzymuje zmienną `closest_so_far` i przy każdym kolejnym obiekcie zawęża do niej górny zakres poszukiwań. W efekcie obiekt zasłonięty (leżący dalej) zostaje odrzucony, a w `HitRecord` ostatecznie ląduje przecięcie najbliższe kamerze.

Same metody `hit` poszczególnych kształtów liczą przecięcie analitycznie i po znalezieniu trafienia wypełniają `HitRecord`: zapisują punkt, ustawiają normalną zwróconą ku promieniowi oraz wskaźnik na materiał obiektu.

5 Jak materiał steruje promieniem

Reakcję powierzchni na światło realizuje rodzina klas dziedziczących po `Material`, a wybór właściwej zachowuje się automatycznie dzięki polimorfizmowi – wywołanie `rec.mat->scatter(...)` trafia do implementacji odpowiadającej rzeczywistemu materiałowi:

- `Diffuse` ustawia `scattered` na promień w losowym kierunku (suma normalnej i losowego wektora) i przyjmuje `attenuation` równe swojemu albedo;
- `Mirror` liczy kierunek odbity i opcjonalnie dodaje drobne rozmycie;
- `Dielectric` wybiera między odbiciem a załamaniem i ustawia białe `attenuation`.

Każda z tych metod zwraca `true`/`false`, informując `ray_color`, czy promień ma być śledzony dalej.

6 Światło bezpośrednie i cienie

Cienie i oświetlenie od źródeł punktowych obsługuje funkcja `direct_light`. Dla trafionego punktu przegląda ona listę świateł sceny i dla każdego liczy kierunek oraz odległość do źródła. Aby sprawdzić, czy punkt jest oświetlony, tworzy pomocniczy promień (promień cienia) i ponownie woła `scene.hit` - jeśli coś przecina drogę do światła, wkład jest pomijany (punkt w cieniu). Warto zauważyć, że kod nie potrzebuje tu osobnego mechanizmu: do testu zasłonięcia wykorzystuje tę samą funkcję wyszukiwania trafień, co przy zwykłym śledzeniu. Wynik `direct_light` dodawany jest w `ray_color` do barwy z odbić.

7 Organizacja i zarządzanie pamięcią

Spójność całości zapewniają dwie klasy abstrakcyjne. `Hittable` narzuca każdemu kształtowi metodę `hit`, a `Material` - metodę `scatter`; `ray_color` i `Scene` operują wyłącznie na wskaźnikach do tych interfejsów. Dlatego dodanie nowego kształtu lub materiału sprowadza się do napisania jednej klasy. Wszystkie obiekty przechowywane są jako `std::shared_ptr` w wektorze sceny, więc pamięć zwalniana jest automatycznie - w kodzie nie ma żadnego `new` ani `delete`.

8 Wynik

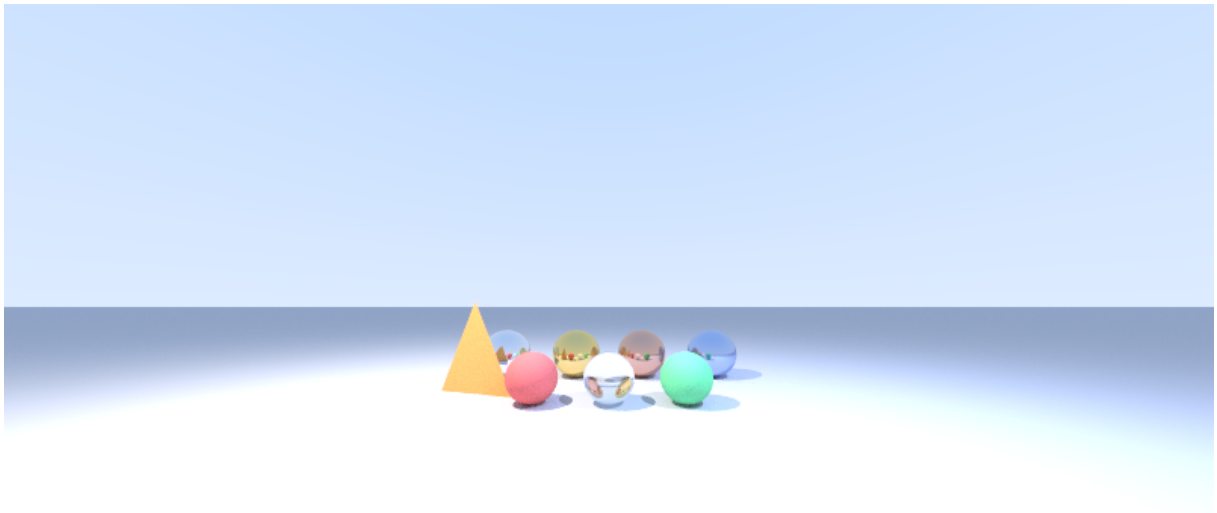


Figure 1: Obraz wygenerowany przez program (800×400, 50 promieni na piksel): odbicia w sferach lustrzanych, załamanie w sferze szklanej, matowe obiekty oraz cienie rzucane przez punktowe źródła światła.

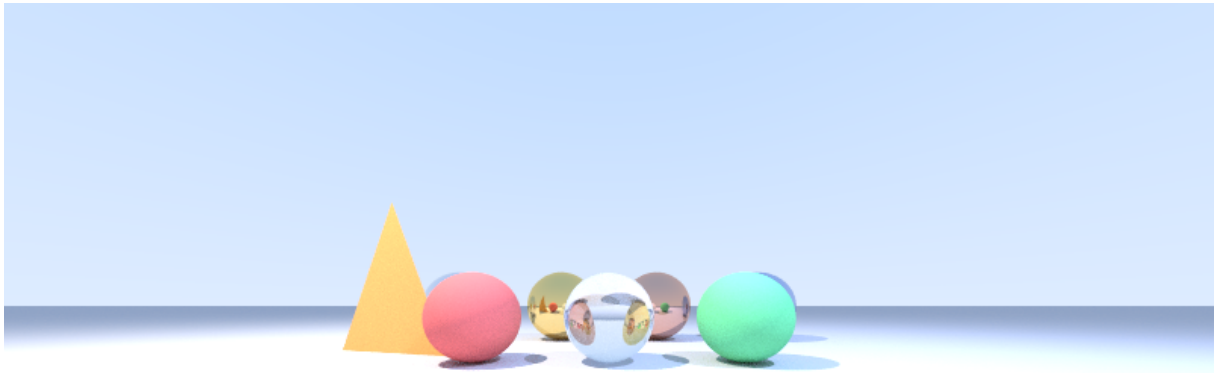


Figure 2: To samo dla innego ustawienia kamery.